

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа № 1

По дисциплине «Обработка изображений
в интеллектуальных системах»

Тема «Обучение классификаторов
средствами библиотеки PyTorch»

Вариант 5

Выполнил
Студент 4 курса
Группы ИИ-25
Дулько Денис Андреевич

Проверил
Андренко К. В.

Брест 2026

Цель работы

Научиться конструировать нейросетевые классификаторы и выполнять их обучение на известных выборках компьютерного зрения.

Задание по варианту 5

Создать простую свёрточную сеть, обучить её на размеченной части STL-10 оптимизатором SGD, оценить точность, построить график ошибки и реализовать распознавание произвольного изображения. Сопоставить результат с опубликованными SOTA-результатами.

Данные и подготовка

STL-10 содержит 10 классов цветных изображений 96×96 : самолёт, птица, автомобиль, кошка, олень, собака, лошадь, обезьяна, корабль и грузовик. Используется `torchvision.datasets.STL10`. Из 5000 размеченных обучающих изображений выделены 4000 для обучения и 1000 для валидации: по 400 и 100 изображений каждого класса. Seed равен 5; разделение в обеих лабораторных одинаковое. Неразмеченные данные в обучение не включены.

Полная тестовая выборка из 8000 изображений используется только для итоговой оценки. Лучшие веса выбираются по минимальной ошибке на валидации. Графики показывают ошибку обучения и валидации, поэтому тест не влияет на выбор эпохи.

Архитектура и обучение

Архитектура: три блока $\text{Conv2d } 3 \times 3 \rightarrow \text{ReLU} \rightarrow \text{MaxPool2d } 2 \times 2$, число каналов 32, 64, 128. Затем $\text{AdaptiveAvgPool2d } 4 \times 4$, Flatten, Linear 2048 $\rightarrow$ 128, ReLU, Dropout 0.3, Linear 128 $\rightarrow$ 10. Все веса обучаются с нуля. На выходе 10 логитов; softmax используется только при предсказании.

Обучающие изображения получают случайную обрезку с `padding=8` и горизонтальное отражение. Значения нормализуются с `mean=std=(0.5, 0.5, 0.5)`; на валидации и тесте случайные преобразования отключены.

SGD: `lr=0.01`, `momentum=0.9`, `weight_decay=0.0001`; `CosineAnnealingLR` уменьшает шаг обучения. Функция ошибки `CrossEntropyLoss`, `batch_size=64`, число эпох 25. Вычисления выполнены на NVIDIA GeForce RTX 3070, PyTorch 2.8.0+cu128.

Результаты обучения

На полной тестовой выборке точность равна **52.49%** средняя ошибка CrossEntropyLoss **1.2763**. Выбрана эпоха 24 по ошибке валидации. Число параметров модели 356 810; обучаемых 356 810. Время обучения и итоговой оценки 0.9 мин.

Эпоха	Train loss	Val loss	Train acc %	Val acc %
1	2.2962	2.2759	11.62	13.90
2	2.2308	2.0746	17.20	24.70
3	2.0444	1.9148	24.65	26.60
4	1.9278	1.7813	28.75	34.10
5	1.8364	1.7269	31.90	35.80
6	1.7549	1.6388	34.10	34.60
7	1.6778	1.5919	36.00	39.90
8	1.6246	1.5415	38.65	40.80
9	1.6114	1.5009	38.83	43.60
10	1.5352	1.4800	43.10	44.10
11	1.5028	1.4440	43.48	45.30
12	1.4598	1.4826	45.15	44.80
13	1.4561	1.4001	44.92	45.50
14	1.4402	1.3692	46.23	47.10
15	1.3875	1.3615	47.67	49.30
16	1.3785	1.3593	48.25	47.50
17	1.3635	1.3344	49.08	49.90
18	1.3266	1.3173	50.40	50.20
19	1.3101	1.3217	51.65	49.30
20	1.2975	1.2859	51.90	51.90
21	1.2859	1.2991	52.10	52.20
22	1.2756	1.2800	51.88	53.40
23	1.2694	1.2754	52.25	52.10
24	1.2699	1.2749	52.42	52.70
25	1.2604	1.2760	53.20	52.60

Графики и предсказания на тестовой выборке

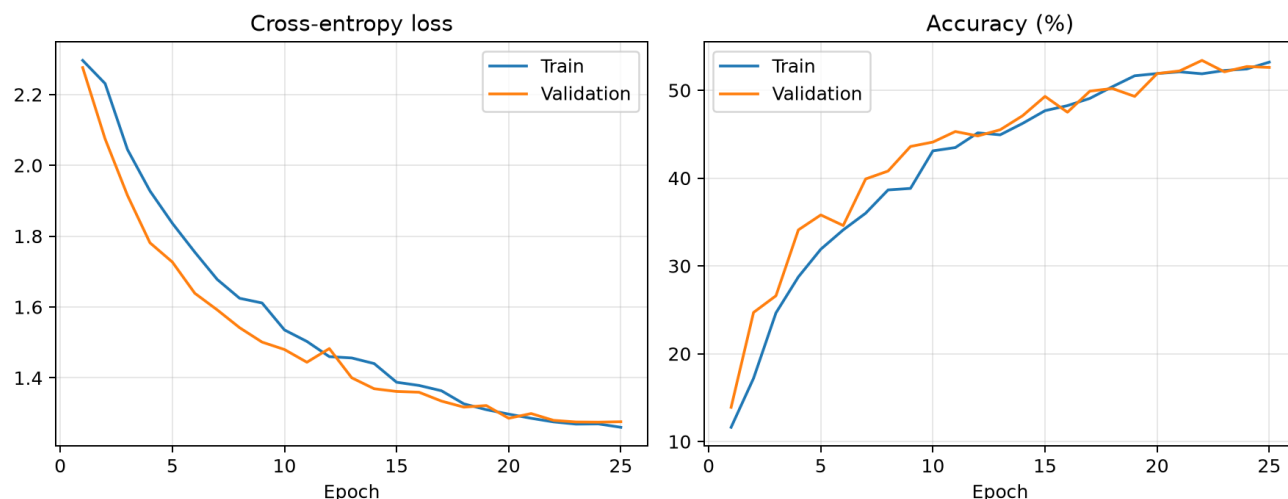


Рисунок 1 Изменение CrossEntropyLoss и точности по эпохам

Train - обучение; Validation - валидация. Случайные аугментации применяются только при обучении. Итоговые тестовые показатели измеряются для сохранённых лучших весов.

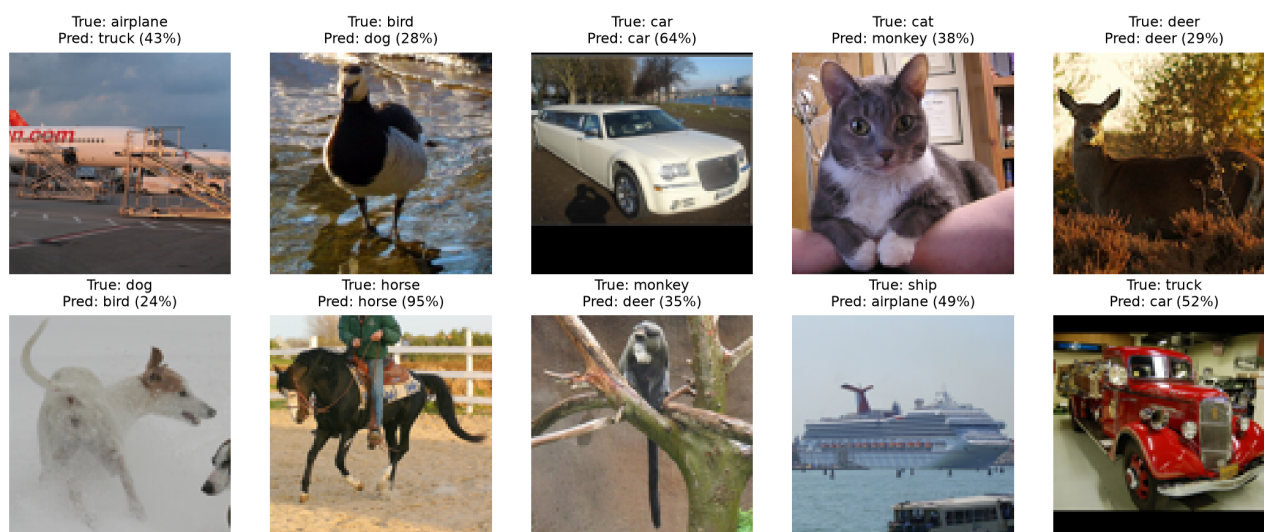


Рисунок 2 По одному тестовому изображению каждого класса

True - настоящий класс; Pred - ответ модели. Показаны первые изображения каждого класса в исходном порядке тестовой выборки, без отбора только правильных ответов. Процент в скобках - значение softmax для предсказанного класса.

Распознавание произвольного изображения

Для проверки выбран внешний файл dog.jpg из открытого репозитория PyTorch Hub. Он не взят из тестовой выборки STL-10. Программа переводит изображение в RGB, приводит к размеру 96×96 и выполняет ту же нормализацию, что на тесте.

Prediction: cat; softmax: 32.4%



Рисунок 3 Ответ собственной CNN на внешнюю фотографию

Ответ CNN: **cat**, значение softmax 32.37%. На фотографии изображена собака; предсказание ошибочно, что показывает ограниченность обобщения простой сети. Softmax не является гарантией правильности.

Запуск программы

```
python reports/Dulko/lab1/src/lab1.py --image reports/Dulko/examples/dog.jpg
```

Параметр `--choose` открывает окно выбора файла. Результат сохраняется в папке `results` в формате PNG; обученные веса загружаются без повторного обучения. Полные команды установки и запуска приведены в README.md.

Сопоставление с опубликованными SOTA результатами

Метод	Точность	Условия
MixMatch [4]	94.41%	5000 меток и неразмеченные данные
ViT-L/16 + Spinal FC + Background [5]	$99.71 \pm 0.06\%$	Предобучение и дополнительные фоновые изображения

Для MixMatch ошибка 5.59% переведена в точность: $100 - 5.59 = 94.41\%$. В статье Kabir результат ViT обозначен как SOTA на май 2023 года. Эти числа являются опубликованными ориентирами, а не утверждением о текущем абсолютном рекорде.

Условия исследований отличаются от лабораторной: используются более мощные сети, внешнее предобучение или дополнительные данные. В этой работе CNN обучается с нуля, а DenseNet использует ImageNet; обучение на STL-10 ограничено 4000 размеченных изображений. Прямое сравнение с SOTA не является сравнением при одинаковых условиях.

Вывод

В ходе работы реализована простая свёрточная сеть и выполнено её обучение с помощью SGD на размеченной части STL-10. Получена тестовая точность 52.49%. Построены графики ошибки и точности, реализована классификация выбранного изображения. Небольшая сеть и ограниченный объём обучения уступают опубликованным моделям; возможные направления улучшения - более длительное обучение и перенос предобученных признаков.

Источники

1. STL-10, описание данных и протокол оценки
<https://cs.stanford.edu/~acoates/stl10/>
2. Torchvision, STL10
<https://docs.pytorch.org/vision/0.23/generated/torchvision.datasets.STL10.html>
3. Torchvision, DenseNet121 и предобработка
<https://docs.pytorch.org/vision/0.23/models/generated/torchvision.models.densenet121.html>
4. Berthelot et al. MixMatch, NeurIPS 2019, таблица 2
<https://papers.nips.cc/paper/2019/file/1cd138d0499a68f4bb72bee04bbec2d7-Paper.pdf>
5. Kabir. Reduction of Class Activation Uncertainty with Background Information, таблицы I и III
<https://arxiv.org/abs/2305.03238>
6. Фотография собаки, PyTorch Hub
<https://github.com/pytorch/hub/blob/master/images/dog.jpg>

Дата обращения к источникам 30.09.2026.